Orbital Dynamics and Stellar Irradiation of TRAPPIST-1e Using Forward Euler and Runge–Kutta Methods
ASTR-UN3273: High-Energy Astrophysics

Madeline Maldonado Gutierrez
Barnard College of Columbia University
(Dated: May 14, 2026)

I. INTRODUCTION

An exoplanet, often called an extrasolar planet, is defined as a planetary body orbiting a star outside the solar system. These planets come in various sizes and compositions, with some resembling those found within our solar system. While they can be discovered using number of methods, the transit method is the most common. A transit occurs when the planet passes in front of the star creating periodic decreases in brightness over time, creating s a light curve plot (see Figure 1).

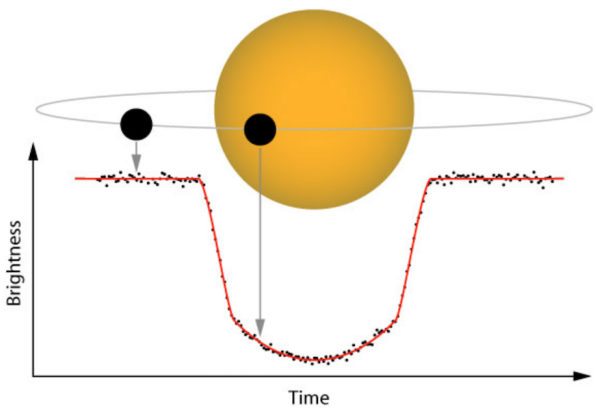

FIG. 1. Simplified diagram of the transit method of a star–planet system [1]

One particularly important type of host stars are M-dwarf stars, which are low-mass ($0.1\ M_\odot < \mathrm{M} < 0.8\ M_\odot$), cool ($2{,}400\ \mathrm{K} < T_{\mathrm{eff}} < 3{,}900\ \mathrm{K}$), and relatively small main sequence stars that make up the majority of stars (approximately 70%) in the Milky Way galaxy [2]. Due to their small radii ($0.09\ R_\odot$) and lower luminosities ($L \leq 0.05\ L_\odot$), planets orbiting M-dwarfs are often located much closer to their host stars compared to planets in Sun-like systems. As a result, these planets can experience strong stellar radiation environments and significant gravitational interactions over relatively short orbital periods.

In this project, the exoplanet TRAPPIST-1e was selected as a representative M-dwarf exoplanet system. TRAPPIST-1e is an Earth-sized exoplanet orbiting the ultracool M-dwarf star TRAPPIST-1 ($\sim 0.09\ M_\odot$) and is one of seven known planets in the system discovered and confirmed using the transit method (see Figure 2). Even though the planets in the TRAPPIST-1 system have relatively short orbital periods of about 1.5 to 19 days, their gravitational interactions build up slowly over time, making long-term observations necessary to accurately constrain the system's orbital dynamics [4].

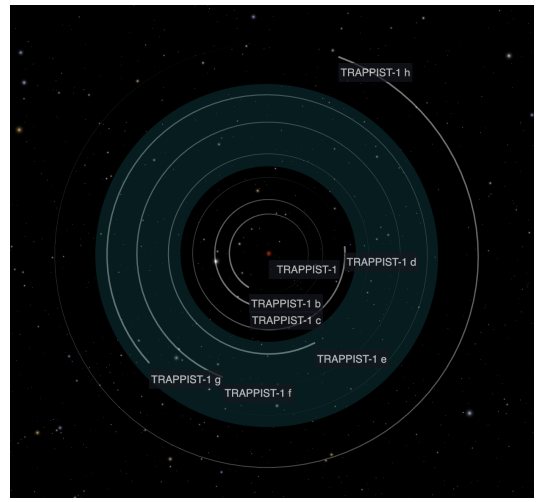

FIG. 2. TRAPPIST-1 System [3]

The planet lies near the star's habitable zone and has been widely studied due to its potential relevance for planetary atmospheres and habitability. Due to the TRAPPIST-1 planets orbiting very close to their host star ($\sim 0.12\ R_\odot$), they provide useful systems for studying orbital dynamics and stellar irradiation [4].

The orbital motion of TRAPPIST-1e was modeled as a two-body gravitational system. The equations of motion were rewritten as a set of coupled first-order ordinary differential equations (ODE) and solved numerically over ten orbital periods. The primary objective of this project was to compare the performance of the first-order Forward Euler (FE) method and the fourth-order Runge–Kutta (RK4) method in terms of orbital stability, numerical accuracy, and conservation of energy and angular momentum.

II. METHODS

The orbital motion of TRAPPIST-1e around its host M-dwarf star TRAPPIST-1 was modeled as a two-body gravitational system in the x-y plane. The exoplanet was treated as a very light test particle moving under the gravitational influence of the central star. Newton's law of gravitation was rewritten as a set of four coupled first-order ODEs describing the position and velocity components of the orbit

$$\dot{x} = v_x \qquad \dot{y} = v_y \tag{1}$$

$$\dot{v}_x = -\frac{GMx}{(x^2+y^2)^{3/2}} \qquad \dot{v}_y = -\frac{GMy}{(x^2+y^2)^{3/2}} \tag{2}$$

where x and y are the orbital coordinates, v_x and v_y are the velocity components, G is the gravitational constant, and $M \approx 0.0898$ $M_\odot$ is the mass of the host star, TRAPPIST-1 [4]. Initial conditions corresponding to circular orbital motion were used. The orbital radius was taken to be the semi-major axis of TRAPPIST-1e (0.02925 AU), and the initial tangential velocity was calculated using

$$v = \sqrt{\frac{GM}{r}} \tag{3}$$

where the orbital radius was defined as $r = \sqrt{x^2+y^2}$, and the gravitational acceleration component was computed using the inverse-square gravitational force law (-GM/r^2).

The equations were numerically integrated over ten orbital periods using both the first-order FE and the RK4 methods and orbital analysis implemented using NumPy in Python (see Appendix A,B, and C). Initial conditions corresponding to circular orbital motion were used, and the simulation was integrated over ten orbital periods using a fixed timestep at $dt = 1000$ seconds.

At each timestep, the orbital trajectory, specific orbital energy was computed from

$$E = \frac{1}{2}(v_x^2 + v_y^2) - \frac{GM_\star}{r} \tag{4}$$

while the specific angular momentum was determined from

$$L = xv_y - yv_x \tag{5}$$

Relative errors in energy and angular momentum were analyzed to compare how effectively each integrator conserved these quantities over time.

$$\frac{Q - Q_0}{|Q_0|} \tag{6}$$

$$\text{Relative Error} = \frac{Q(t) - Q_0}{|Q_0|} \tag{7}$$

where Q represents either the specific orbital energy or angular momentum.

The time-dependent stellar irradiation received by TRAPPIST-1e was estimated using the inverse-square law for flux

$$F \ \alpha \ \frac{1}{r^2} \tag{8}$$

The stellar irradiation was normalized relative to the initial orbital distance using

$$\frac{F}{F_0} = (\frac{r_0}{r})^2 \tag{9}$$

where r_0 is the initial radius and r is the instantaneous orbital distance from the host star.

Fourth-Order Runge-Kutta Method

The RK4 algorithm evaluates four intermediate slopes during each timestep and combines them through a weighted average to produce a fourth-order accurate estimate of the next state vector. It has a global truncation error order $O(h^4)$, making it substantially more accurate than the first order FE method ($O(h)$) for long-term integrations. The integrator was implemented through a reusable RungeKutta4.step() function that evolved the orbital state vector $[x, y, v_x, v_y]$ forward in time, representing the planet's position and velocity components in the orbital plane.

First-Order Forward Euler Method

For comparison, the same orbital system was integrated using the first-order FE method. The Euler integrator advanced the state vector using only the local derivative evaluated at the current timestep according to

$$y_{n+1} = y_n + hf(t_n, y_n) \tag{10}$$

Unlike RK4, the Euler method does not account for changes in the trajectory curvature within each timestep, leading to significantly larger truncation errors during long integrations.

III. RESULTS

Orbital trajectories generated by both numerical methods were compared over ten orbital periods. The orbital state vectors were stored at every timestep to analyze trajectories (see Appendix A and B). The RK4 integrator produced a stable closed orbit (shown in purple) with nearly constant orbital radius, while the first-order FE method (shown in blue) showed a progressively expanding spiral trajectory (see Figure 3). The outward drift observed in the Euler orbit resulted from accumulated numerical error that produced a non-physical increase in the orbital energy. As a result, the simulated planet gradually moved farther from the host star (shown in red) over time instead of maintaining a stable closed orbit expected for a true two-body gravitational system.

The RK4 method maintained energy conservation to a very high degree throughout the simulation, while the

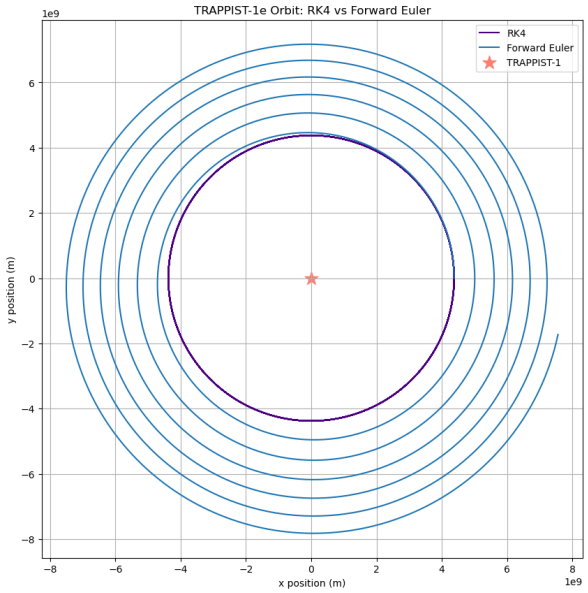

FIG. 3. A comparison of the RK4 and FE methods TRAPPIST-1e orbit around TRAPPIST-1 over ten orbital periods

first-order FE method showed a continuously increasing energy error (see Figure 4).

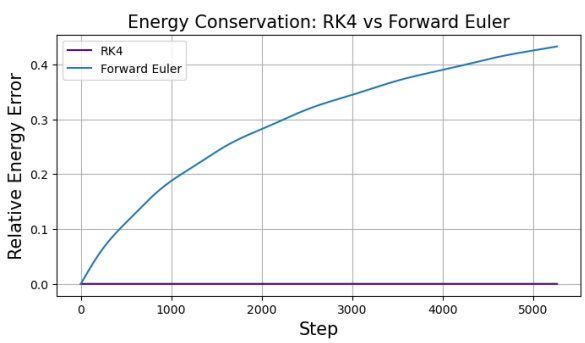

FIG. 4. Energy conservation for the TRAPPIST-1e orbit around TRAPPIST-1 over ten orbital periods using RK4 and FE methods

The RK4 integrator preserved angular momentum extremely well over the ten simulated orbits, whereas the first-order FE method showed substantial angular momentum drift (see Figure 5). This loss of angular momentum conservation contributed directly to the orbital instability observed in the Euler trajectory.

TRAPPIST-1e and M-dwarf Irradiation

The stellar flux was normalized relative to the initial orbital position and tracked throughout the simulation. Since the RK4 orbit maintained a nearly constant orbital radius, the irradiation remained essentially constant over time. In contrast, the outward spiral generated by the FE method caused the orbital radius to increase artificially, resulting in a steadily decreasing irradiation profile (see

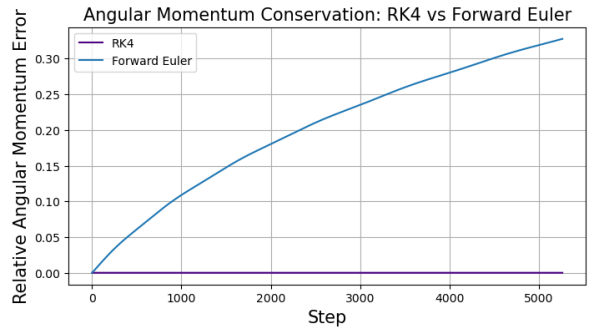

FIG. 5. Angular momentum for the TRAPPIST-1e orbit around TRAPPIST-1 over ten orbital periods using RK4 and FE methods

Figure 6). This demonstrated that numerical integration errors can propagate into derived astrophysical quantities such as stellar radiation exposure in M-dwarf exoplanet systems.

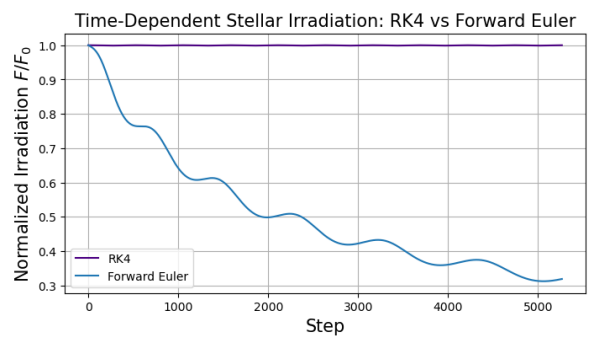

FIG. 6. A time-dependent stellar irradiation of TRAPPIST-1e orbit around TRAPPIST-1 over ten orbital periods using RK4 and FE methods

IV. CONCLUSION

This project compared the performance of the first-order Forward Euler and fourth-order Runge–Kutta methods for modeling the orbit of TRAPPIST-1e around its M-dwarf host star. The RK4 integrator produced a stable closed orbit while conserving energy and angular momentum to a high degree throughout the simulation. In contrast, the Forward Euler method accumulated significant numerical error, resulting in nonphysical orbital expansion and large deviations in conserved quantities. These numerical errors also propagated stellar irradiation, demonstrating that integration accuracy plays an important role in simulations of exoplanet environments and high-energy stellar exposure.

Appendix A: First-Order Forward Euler Code

```
def euler_step(f, t, y, h):
    """
    First-order Forward Euler integrator for
    systems of
```

```
    first-order ordinary differential equations (ODEs).

    The Forward Euler method advances the solution using
    the local derivative evaluated at the current timestep:

        y_(n+1) = y_n + h*f(t_n, y_n)

    where:
        y_n: current state vector
        h: timestep size
        f(t, y): derivative function (RHS of the ODE system)

    Notes:
        - This is an explicit first-order numerical method.
        - The method is simple and computationally inexpensive,
          but accumulates truncation errors over long integrations.
        - For orbital motion problems, these errors can produce
          nonphysical changes in quantities such as energy and
          angular momentum.

    Args:
        f (callable): function that computes dy/dt for the system.
        t (float): Current simulation time.
        y (np.ndarray): Current state vector.
        h (float): Timestep size.

    Returns:
        np.ndarray: Updated state vector after one Euler step.
    """
    return y + h * f(t, y)
```

Appendix B: Fourth-Order Runge-Kutta Code

```
class RungeKutta4:
    """
    Runge-Kutta 4th-order stepper (RK4) for first-order ordinary differential equations (ODEs).
    systems dy/dr = f(t, y)

    Provides:
        - step(f, r0, y0, h): advance one RK4 step from (r0, y0) to (r0 + h, y1)
        - orbit_rhs(t, y): RHS for orbital motion with state vector [x, y, vx, vy]
    """
    @staticmethod
    def step(f, r0, y0, h):
        """
        Performs RK4 for the average of four vector directions in first-order
        forward Euler steps, respectively. This function counts the number of iterations
        using step Euler size or step height h

        Args:
            f (callable): function that computes dy/dt at (t0, y0)
            r0 (float): current value of the independent variable, here time
            y0 (np.ndarray): current state vector
            h (float): timestep size

        Returns:
            np.ndarray: updated state after one RK4 Euler step (approx. to y at r0 + h)
        """
        k1 = f(r0, y0) # at base point
        k2 = f(r0 + 0.5*h, y0 + 0.5*h*k1) # evaluate state vector after forward euler step in k1
        k3 = f(r0 + 0.5*h, y0 + 0.5*h*k2) # evaluate state vector after forward euler step in k2
        k4 = f(r0 + h, y0+h*k3) # evaluate state vector after ffull euler step in k3

        # avg. of the four vector directions for 4th-order accuracy
        result = y0 + (h/6.0)*(k1 + 2*k2 + 2*k3 + k4) # update next value of y
        return result

    @staticmethod
    def orbit_rhs(t, y):
        """
        Right-hand side (RHS) of the two-body orbital motion equations
        for the TRAPPIST-1e system in the x-y plane.

        The state vector is defined as:
            y = [x, y_pos, vx, vy]

        where:
            x, y_pos: orbital position coordinates (m)
            vx, vy: velocity components (m/s)

        The function computes the derivatives:
            dx/dt  = vx
            dy/dt  = vy
            dvx/dt = ax
            dvy/dt = ay

        using Newtonian gravity for a test particle orbiting
        a central M-dwarf star.

        Notes:
            - The orbital radius is computed as:
                r = sqrt(x^2 + y^2)

            - Gravitational acceleration follows the inverse-square law:
                a = -GM/r^2

            - The acceleration components are:
                ax = -GMx/r^3
                ay = -GMy/r^3

        Args:
            t (float): Current simulation time (s).
            y (np.ndarray): Current state vector [x, y_pos, vx, vy].

```

```python
        Returns:
            np.ndarray: Array containing the time derivatives: [vx, vy, ax, ay].
        """
        x, y_pos, vx, vy = y

        G = 6.67430e-11
        M_star = 0.0898 * 1.98847e30 # mass of star in kg (0.0898 solar masses)

        r = np.sqrt(x**2 + y_pos**2)

        ax = -G * M_star * x / r**3
        ay = -G * M_star * y_pos / r**3

        return np.array([vx, vy, ax, ay], dtype=float)
```

Appendix C: TRAPPIST-1e Orbit Analysis Code

```python
# constants
G = 6.67430e-11
M = 0.0898 * 1.989e30

# TRAPPIST-1e initial position conditions
x0 = 0.02925 * 1.496e11   # semi-major axis: AU -> meters
y0 = 0.0 # chosen

vx0 = 0.0
vy0 = np.sqrt(G * M / x0)

# initial state for the first order forward ruler orbit
state = np.array([x0, y0, vx0, vy0], dtype=float)

# orbital period
orbital_period = 2 * np.pi * np.sqrt(x0**3 / (G * M))

# integration setup
dt = 1000 # step size in seconds
T_total = 10 * orbital_period
N_steps = int(T_total / dt)

# FE integration
trajectory_euler = []

state_euler = np.array([x0, y0, vx0, vy0], dtype=float)
t = 0.0

for i in range(N_steps):
    trajectory_euler.append(state_euler.copy())
    state_euler = euler_step(RungeKutta4.orbit_rhs, t, state_euler, dt)
    t += dt

trajectory_euler = np.array(trajectory_euler)

# RK4 integration
trajectory_rk4 = []

state_rk4 = np.array([x0, y0, vx0, vy0], dtype=float)
t = 0.0

for i in range(N_steps):
    trajectory_rk4.append(state_rk4.copy())
    state_rk4 = RungeKutta4.step(RungeKutta4.orbit_rhs, t, state_rk4, dt)
    t += dt

trajectory_rk4 = np.array(trajectory_rk4)

# RK4 relative energy error
x_rk4 = trajectory_rk4[:, 0]
y_rk4 = trajectory_rk4[:, 1]
vx_rk4 = trajectory_rk4[:, 2]
vy_rk4 = trajectory_rk4[:, 3]

r_rk4 = np.sqrt(x_rk4**2 + y_rk4**2)

energy_rk4 = 0.5*(vx_rk4**2 + vy_rk4**2) - G*M/r_rk4

energy_error_rk4 = (
    (energy_rk4 - energy_rk4[0])
    / abs(energy_rk4[0])
)

# FE relative energy error
x_euler = trajectory_euler[:, 0]
y_euler = trajectory_euler[:, 1]
vx_euler = trajectory_euler[:, 2]
vy_euler = trajectory_euler[:, 3]

r_euler = np.sqrt(x_euler**2 + y_euler**2)

energy_euler = 0.5*(vx_euler**2 + vy_euler**2) - G*M/r_euler

energy_error_euler = (
    (energy_euler - energy_euler[0])
    / abs(energy_euler[0])
)

# RK4 relative angular momentum error
x_rk4_m = trajectory_rk4[:, 0]
y_rk4_m = trajectory_rk4[:, 1]
vx_rk4_m = trajectory_rk4[:, 2]
vy_rk4_m = trajectory_rk4[:, 3]

L_rk4 = x_rk4_m * vy_rk4_m - y_rk4_m * vx_rk4_m

L_error_rk4 = (
    (L_rk4 - L_rk4[0])
    / abs(L_rk4[0])
)

# FE relative angular momentum error
x_euler_m = trajectory_euler[:, 0]
y_euler_m = trajectory_euler[:, 1]
vx_euler_m = trajectory_euler[:, 2]
vy_euler_m = trajectory_euler[:, 3]

L_euler = x_euler_m * vy_euler_m - y_euler_m * vx_euler_m

L_error_euler = (
    (L_euler - L_euler[0])
    / abs(L_euler[0])
)

# Distance from star
r_rk4 = np.sqrt(x_rk4**2 + y_rk4**2)
r_euler = np.sqrt(x_euler**2 + y_euler**2)

# Normalized irradiation: F/F0 = (r0/r)^2
```

```
flux_rk4 = (r_rk4[0] / r_rk4)**2
flux_euler = (r_euler[0] / r_euler)**2
```


[1] Transiting Exoplanet Survey Satellite (TESS)
[2] Bochanski, J. J. et al. 2010, AJ, 139, 2679
[3] NASA Eyes on Exoplanets, TRAPPIST-1 System
[4] Agol, E. et al. 2021, Planet. Sci. J. 2 1
[5] Gillon, M., Triaud, A., Demory, BO. et al. Seven temperate terrestrial planets around the nearby ultracool dwarf star TRAPPIST-1. Nature 542, 456–460 (2017)